COMP2004 · TERM PROJECT

Bug Tracking System

Database design, schema, and live MySQL demo.

Presented by **MEGAMIND**
Instructor: Dr. Pınar Efe
Fenerbahçe University · Computer Engineering

The problem

Software teams ship bugs. They need a system to track every defect from report to closure — across multiple projects, with audit trails.

01

Multi-project

Multiple projects run in parallel. One user can be Manager in one project, Developer in another.

02

Full lifecycle

Bug states: Open → InProgress → Resolved → Closed → Reopened. With reassignments at any point.

03

Audit trail

Every change to status, priority, severity, or assignee writes a row to an audit table.

Result: **8 entities, 11 relationships, 3NF.**

What the system must do

Users & Access

Global roles + per-project roles. Authenticated by unique email.

Assignment

At most one current assignee per bug. Full reassignment history preserved.

Projects

Multi-project. Each has owner + members with project-specific roles.

Audit

Every change to status / priority / severity / assignee is logged.

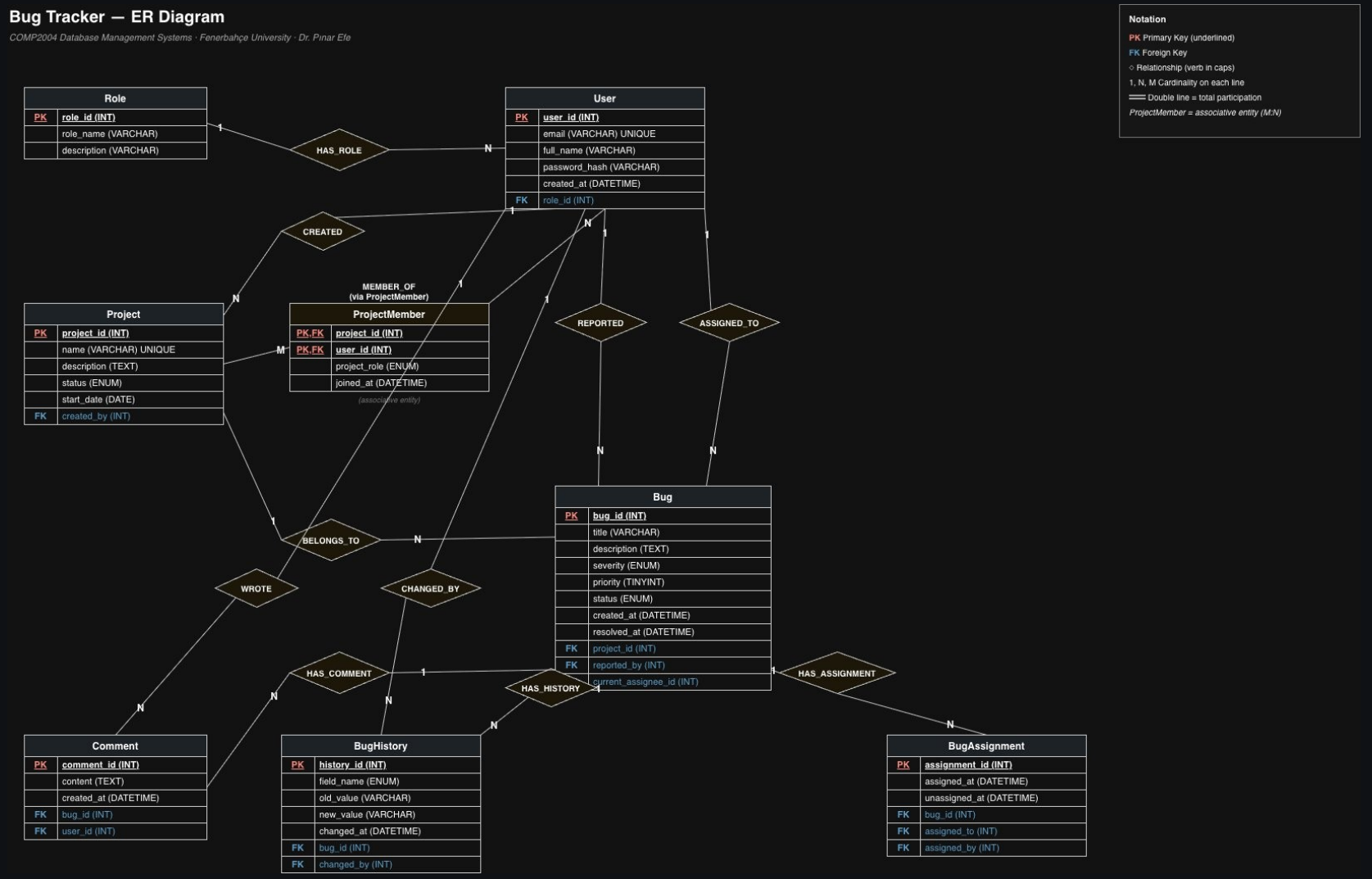
Bug Reporting

Members can report bugs against their projects. Severity, priority, status.

Reporting

Bug aging, top reporters, avg resolution time, bugs per developer.

Entity-Relationship diagram



Notation

8 ENTITIES

11 RELATIONSHIPS

PK Primary key (underlined)
FK Foreign key
◇ Relationship (verb in caps)
1, N, M Cardinality at endpoints

ProjectMember
Associative entity (M:N) with own attributes — must be its own table.

8 tables, mapped from the ER

All MySQL 8 / InnoDB. Constraints declarative.

#01

Role

`role_id · role_name · description`

#02

User

`user_id · email* · full_name · role_id`

#03

Project

`project_id · name* · status · created_by`

#04

ProjectMember

`(project_id, user_id) · project_role`

#05

Bug

`bug_id · severity · priority · status · …`

#06

BugAssignment

`assignment_id · assigned_to · assigned_at`

#07

Comment

`comment_id · content · bug_id · user_id`

#08

BugHistory

`history_id · field_name · old/new value`

`*` = *UNIQUE constraint*

Three calls I'll defend in Q&A

01

Why is ProjectMember a separate table?

M:N relationships always become tables when mapped from ER. AND it has own attributes (project_role, joined_at) — these can't live on User or Project alone.

02

Why does User connect to Bug twice?

REPORTED ≠ ASSIGNED_TO. Reporter is set at insert and never changes — assignee changes over time. Different semantics, different lifecycles, different FK columns.

03

Why both current_assignee_id AND BugAssignment?

current_assignee_id is an O(1) read for the UI. BugAssignment is the audit trail. Without the FK, every list query needs JOIN + ORDER BY DESC LIMIT 1.

Why this schema is 3NF

3NF, step by step

1NF

Atomic attributes

No repeating groups, no array columns. Every cell is a single value.

2NF

Full dependency on whole key

ProjectMember's PK is composite (project_id, user_id). project_role and joined_at depend on BOTH.

3NF

No transitive dependencies

Role names live in Role, not duplicated in User. No non-prime → non-prime.

FK delete strategy

CASCADE `Project → Bug`

project gone = its bugs gone

RESTRICT `User → Bug.reported_by`

preserve audit trail

SET NULL `User → Bug.assignee`

user leaves = unassigned

CASCADE `Bug → trail tables`

no bug = no trail

RESTRICT `Role → User`

no orphan users

200 records · 20 queries

200	10	5	5
TOTAL ROWS	DML STATEMENTS	SIMPLE QUERIES	COMPLEX QUERIES

Example complex query — bug aging report

Business question: For every Open bug older than 14 days, show project, reporter, assignee (or 'Unassigned'), age in days.

```
SELECT b.bug_id, p.name AS project, u_rep.full_name AS reporter,
       COALESCE(u_asg.full_name, 'Unassigned') AS assignee,
       DATEDIFF(CURDATE(), b.created_at) AS days_open
FROM Bug b
JOIN Project p ON b.project_id = p.project_id
JOIN User u_rep ON b.reported_by = u_rep.user_id
LEFT JOIN User u_asg ON b.current_assignee_id = u_asg.user_id
WHERE b.status = 'Open' AND DATEDIFF(CURDATE(), b.created_at) > 14
ORDER BY days_open DESC;
```

Live demo, in 6 steps

1 min

Open MySQL Workbench

Reverse-engineer the schema. Show 8 tables in the navigator.

1 min

Inspect a table

Open Bug. Point out PK, FKs, CHECK on priority, ENUM on status.

2 min

Run simple queries

S-1 (open critical bugs) and S-4 (unassigned by priority) — quick wins, real data.

2 min

Run DML

Insert a new bug (DML-3) → assign to a developer (DML-4) → see the row update.

3 min

Run complex queries

C-1 bugs by severity per project · C-3 avg resolution time · C-5 bug aging report.

1 min

Wrap

Recap 3NF, FK delete strategy, and where the audit trail lives.

QUESTIONS

Thank you.

Now I'd like to take your questions.

8 entities

11 relationships

200 rows · **20** queries · **3NF**